SPRING DATA REST

What is Spring Data REST?

Spring Data REST is a Spring module that automatically exposes your Repository as REST APIs.

You write Entity

You write Repository

Spring creates REST APIs automatically

No need to write Controller

No need to write Service layer if no processing is needed

👉 Repository itself acts as the Controller

2️⃣ "Every repository becomes an API" — what it means

For each repository, Spring Data REST provides default CRUD endpoints:

GET → fetch data

POST → create data

PUT / PATCH → update data

DELETE → remove data

Example:

ProductRepository → /products
OrderRepository → /orders

So yes:

Every repository automatically becomes a CRUD API

3️⃣ Does Spring Data REST remove Controller and Service layer?

✓ YES, when:

Only basic CRUD

No business logic

No validation

No calculations

No DTOs

Flow:

Client → Spring Data REST → Repository

❌ NO, when:

Service layer is required

Business logic exists

DTOs are required

Custom request/response needed

Flow:

Client → Controller → Service → Repository

👉 Spring Data REST cannot replace service-layer logic

4️⃣ Can we use Spring Data REST with Service layer?

Technically yes, but not recommended.

Reason:

Spring Data REST directly calls repository

Service layer is bypassed

Business rules & DTO mapping break

So:

If Service layer exists → don't rely on Spring Data REST APIs

---

5️⃣ Where do DTOs fit in?

DTOs are used to:

Hide DB structure

Hide IDs / foreign keys

Send UI-friendly responses

Control request/response shape

But:

Spring Data REST exposes entities directly

No clean place for DTO mapping

So:

If DTOs are needed → don't use Spring Data REST

6️⃣ "Disable repository exposure for sensitive entities" — meaning

Because:

Every repository becomes an API

Entities get exposed automatically

Sensitive entities like:

User

Order

Payment

Should:

NOT be exposed directly

Use manual controllers instead

7️⃣ Where Spring Data REST is actually used

Mostly used for:

Admin panels

Internal tools

Simple CRUD apps

Prototypes

Learning/demo projects

Rarely used in:

Large enterprise production systems

8️⃣ What companies usually do in real projects

In real companies:

Controllers are written manually

Service layer is mandatory

DTOs are always used

Repositories are not exposed directly

So:

Learn Spring Data REST for knowledge,
but follow company architecture at work

🧠 **Final mental model (remember this)**

Spring Data REST = Repository → Controller → API (auto-generated)
Service + DTOs present = Write Controller yourself

9️⃣ What are Hateos?

HATEOAS means an API response includes links for all CRUD actions, allowing the client to navigate to related resources without knowing URLs in advance.

That sentence is interview-safe 👍

🧠 **Slightly more detailed (simple words)**

- When you call an API (GET / POST / PUT / DELETE)
- The response does **not only return data**
- It also returns **links**
- These links tell:
  - where to get related data
  - where to update
  - where to delete
  - where to create related resources

So the **client follows links**, instead of guessing endpoints.

📦 Example (GET request)

Request

GET /jobPosts/1

Response (simplified)

```
{
  "postId": 1,
  "postProfile": "Java Developer",

  "_links": {
    "self": {
      "href": "http://localhost:8080/jobPosts/1"
    },
    "jobPosts": {
      "href": "http://localhost:8080/jobPosts"
    }
  }
}
```

What this means

- self → this exact resource
- jobPosts → all job posts
- Client can now navigate without knowing URLs

🔄 How HATEOAS supports CRUD

Operation	How HATEOAS helps
CREATE	Link to collection resource
READ	Link to self & related resources
UPDATE	Link to update current resource
DELETE	Link to delete resource

Spring Data REST **automatically provides these links**.

🧩 Why this is useful

✅ Pros

- Client doesn't hardcode URLs
- API changes don't break clients
- Self-descriptive API
- Easier API evolution

❌ Cons (real world truth)

- Extra response size
- Many frontends ignore it
- Overkill for simple APIs

🧠 One-liner to remember forever

HATEOAS turns an API into a navigable graph instead of a list of fixed URLs.